

A Voxel Game in Dyalog APL

Implementation status, open work, and why it is open

Prepared 7 October 2026. Sources: the repository at `avoxelgame-fork` at commit `770e8d8` (4 October 2026), and the author's write-up *Notes on writing a voxel game in Dyalog APL* (Kyle Croarkin, 6 March 2026), at homewithinnowhere.com/posts/2026-03-06-voxel-game.html.

The project set out to answer one question: can a Minecraft-shaped voxel engine be written in Dyalog APL, in array style, and still hold 60 frames per second. That question is answered yes. Chunk meshing, culling, collision, ray picking, terrain generation and streaming all work and all live in array code. Roughly 2,400 lines of APL and 330 lines of C glue carry the whole engine.

What remains open falls into three groups. Some features were deliberately deferred. Some are blocked by the array paradigm itself, where the author judged the natural APL formulation too slow and declined to escape into C. The rest are platform and interpreter limits outside the project.

What is implemented

Core engine

Area	State	Notes
Chunk meshing	Working	Chunks are 16 by 128 by 16. All possible cube faces are precomputed once, then a boolean mask selects only exposed faces. Vertices are bit-packed into four bytes each.
Visibility culling	Working	Axis-aligned bounding box tested against the six frustum planes, one array expression over all loaded chunks. Two earlier attempts, an angle test and a corner-point test, were replaced.
Collision	Working	Swept bounding box per axis with a Minkowski sum, resolving against the nearest colliding block.
Block break and place	Working	A short ray march samples one hundred points over five units and picks the first solid hit. Placement is rejected when the new block would intersect the player.
Terrain generation	Working	Three-dimensional fractal value noise over three octaves, hashed from the world seed so the terrain is reproducible. Stone, dirt, grass, sand, water and one tree per chunk.
Chunk streaming	Working	View distance of ten, so up to 441 chunks resident. Loads and unloads are capped at two disk operations per frame to stop frame-time spikes.
Save and load	Working	Worlds persist to Dyalog component files. Chunks are run-length packed, indexed by a hashed coordinate table. Player position, orientation, velocity, noclip flag and the seed all persist.

Rendering and platform

- **Graphics backend.** SDL3 with the new SDL GPU API, reached through Dyalog's foreign function interface. Metal on macOS, Vulkan on Linux and Windows. Shaders ship precompiled as Metal, SPIR-V and DXIL, with a shell script to rebuild them from the GLSL sources.

- **C support library.** A small shared library named LSE wraps the parts of SDL that do not express well in APL: tagged-union event polling, nested pipeline creation structs, font linked lists, raw memory copies and batched draw calls.
- **Textures.** A five by five block atlas is sliced into a texture array with mipmaps. Ten block types are defined, each with per-face texture indices.
- **Text.** Font rendering through SDL3_ttf into GPU buffers, with two inverted tables, one for loaded strings and one for what is currently on screen. A checksum avoids regenerating unchanged vertex data.
- **User interface.** Title screen with animated logo, a save-slot picker, an in-world hotbar highlighting the selected block, a crosshair, and a statistics overlay with chunk counts, average frame rate and worst frame rate.
- **Water.** Stored as a distinct negative block value. Only the surface layer is meshed, as upward-facing quads, and the whole sheet rises and falls on a sine wave.
- **Other polish.** Distance fog, live window resizing, noclip flight, mouse grab toggle, interface hide toggle, and a three-state machine covering menu, save picker and world.

Blocked by the array paradigm

These are the two features the author names as genuinely hard in APL, not merely unwritten. Both reduce to the same shape: spread outward from a point until blocked. Expressed over a whole three-dimensional array, the natural formulation is a fixpoint iteration that touches every voxel on every step, which is far too slow to run when a torch is placed or water moves.

Lighting

Not started. There is no light map, no sky light, no block light. The fragment shader applies texture and fog only, so the world is uniformly lit. A conventional engine would breadth-first flood light outward from the sky and from emitters, stopping at solid blocks. The author considers that control-flow-heavy crawl to be outside what APL does well, and writing it in C or a compute shader would, in their words, be cheating on the original premise.

Water simulation

Partially present as a visual, absent as a simulation. Water never flows, spreads or drains. Sea level is a single global constant, so every body of water in the world sits at the same height and animates with the same wave. Only the topmost water layer is meshed, so submerged water is invisible from inside. The blocker is the same per-tick directional spread problem as lighting.

The author has an open invitation on this point, and will accept a solution that works on arrays of 16 by 128 by 16 and benchmarks under two milliseconds. A naive stencil-and-fixpoint sketch is given in the write-up, but it ignores solid blocks blocking the spread.

Blocked outside the project

Issue	Why it is blocked
-------	-------------------

Windows performance regression	Open and acknowledged in the readme. A pipeline error on the DirectX 12 backend forced a rollback to Vulkan on Windows, and view distance was cut to compensate. Backend choice is hardcoded per operating system, so DirectX 12 is unreachable at present.
Cannot play twice in one session	Starting a second world in the same interpreter session raises system error 999. The readme attributes this to leaks that have not been located. Chunk vertex buffers are created and released individually with no pool, which is the likely place to look.
Stencil state cannot be passed	Dyalog's foreign function interface appears unable to pass structs by reference inside structs passed by reference. The stencil operation states are commented out of both the APL binding and the C wrapper, with a note about unexplained padding, leaving the depth and stencil pipeline stage simplified.
Threading abandoned	An isolates branch was tried and dropped. On a low-end laptop the per-isolate overhead outweighed the gain. Two pieces were salvaged into the main line: the worst-frame-rate counter, and freezing player movement while the player's own chunk is still loading.
No single-precision floats	Array languages carry doubles only. Worked around by packing vertex attributes into four unsigned bytes, so this costs nothing today, but it would bite any per-frame float data.
Callback-based APIs ruled out	Dyalog restricts callback functions to one specific Fortran library. This is why SDL GPU was chosen over wgpu-native, whose API relies on callbacks.

Unfinished or deferred by choice

Known rough edges in shipped code

- **Chunk borders are treated as air.** The exposed-face test pads each axis with a zero plane rather than consulting the neighbouring chunk, so every face on a chunk boundary is emitted and drawn even when hidden. This replaced an earlier variant that ignored edge blocks outright. Both are described as suboptimal but acceptable.
- **Two ray-casting routines, the better one unused.** A proper fast voxel traversal exists and is correct, but nothing calls it. The author is dissatisfied with its recursive form and wants a non-recursive version. Block picking uses the crude fixed-step sampler instead, which is only valid over short distances.
- **No vertex buffer pooling or GPU fences.** The write-up describes shared buffer pools and fences for chunks moving on and off the device. The current code allocates and frees a buffer per chunk upload instead. A capacity constant for the pool is still computed but never read.
- **Tree placement is crude.** One tree per chunk at most, placed by a per-chunk seeded random pick, and confined well inside the chunk so it can never cross a boundary. Flagged in a comment as a shoddy way to do it.
- **Save slots are fixed and unnamed.** Four slots, shown oldest to newest, auto-named by index. A comment states this is a stopgap to avoid file conflicts until the player can type a world name. There is no rename and no delete. The save-screen draw routine is an empty stub.
- **Terrain occupies a slice of the column.** Noise fills only a 64-block band, with solid stone below it and air above, so a good part of the 128-block height is unused. A code comment notes the three-dimensional noise costs considerably more than two-dimensional Perlin noise would.

- **Documentation drift.** The readme lists keys 1 through 5 for block selection; the code binds 1 through 6, the sixth being wood planks.

Explicit next steps left in the code and the write-up

- Investigate reading several component file records per frame in a single system call, instead of the current fixed two disk operations per frame. Named in the most recent commit message.
- Replace the shader build shell script with a Dyalog script, now that the project is on Dyalog 20.0.
- Perlin noise for terrain, named in the write-up as the next target alongside map saving, which has since landed.
- Read and work with existing APL code written by others, rather than only the author's own.
- Experiment with Apter trees adapted to quadtrees, as a deliberately harder problem to express in array style.

Not attempted

No inventory or crafting, no entities or creatures, no day and night cycle, no sound, no multiplayer, no block-breaking feedback beyond instant removal, and no automated tests or continuous integration. None of these are blocked. They are simply outside what the experiment set out to prove.

Where to look in the source

Path	Contents
avg/world.apln	The heart of the project. Chunk table, meshing, terrain generation, streaming, disk persistence and frustum culling, in about 520 lines.
avg/player.apln	Camera, frustum derivation, collision resolution, input dispatch, block break and place.
avg/world_state.apln	Per-frame update and draw order, world pipeline creation, save file lifecycle, statistics overlay.
avg/menu_state.apln	Title screen and save picker, including the empty save-screen draw stub.
avg/state.apln	The state machine and the application entry point.
avg/text.apln, avg/world_ui.apln	Text rendering tables and the in-world hotbar.
avg/lse.apln, lse/src/	Foreign function declarations and the C support library, including the disabled stencil binding.
avg/Ray_hit.aplf, avg/Silly_hit.aplf	The unused fast traversal and the crude sampler that is used in its place.

Status in one line: the engine question is settled and the game is playable, lighting and fluid dynamics are the two real walls, and the Windows backend plus the single-session restriction are the two defects a player would notice first.